Guia Completo de Utilizador do ATC

Guia Completo de Utilizador do Advanced Trigonometry Calculator

Versão: ATC 2.1.9 Idioma: Português de Portugal Autor: Renato Alexandre dos Santos Freitas

Este é o guia de utilizador expandido do Advanced Trigonometry Calculator (ATC). Foi construído a partir do guia online existente, da documentação 2.1.8 e das atualizações para 2.1.9 e da cobertura automatizada de regressão deste repositório.

1. O que é o ATC

O ATC é uma aplicação matemática gratuita, open source e de linha de comandos para Windows. Foi desenhada para trabalho matemático local rápido através de comandos e expressões textuais.

Casos tipicos:

- cálculos científicos rápidos;
- trigonometria e trigonometria inversa;
- numeros complexos;
- matrizes;
- estatística e probabilidade;
- funções DSP;
- simplificacao polinomial;
- resolucao de equacoes e sistemas;
- variáveis e expressões reutilizaveis;
- módulos interativos guiados;
- processamento de ficheiros TXT.

2. O que o ATC não é

O ATC foca-se deliberadamente em computação matemática prática orientada por comandos, em vez de tentar ser um CAS universal.

- O ATC não é um CAS geral completo como Mathematica, Maple, SageMath ou SymPy.
- O ATC não foi desenhado para provar teoremas matemáticos arbitrários.
- O ATC não substitui ferramentas profissionais de validação específicas de engenharia.
- O ATC foca-se em cálculo numérico, fluxos por comandos, uso educativo, funcionalidades hibridas simbólicas/numéricas implementadas e execução local rápida.
- Existe comportamento simbólico apenas onde está explicitamente suportado e testado.

3. Quick Start

O ATC é uma aplicação matemática de linha de comandos para Windows, gratuita e open-source. Avalia expressões matemáticas, comandos documentados e workflows guiados. Deve ser entendido como uma calculadora local prática, não como um CAS simbólico universal.

Abrir o ATC executando `atc.exe` a partir da pasta Release criada pelo build ou através do atalho/launcher instalado quando existir. O prompt aceita um comando ou expressão por linha.

Primeiros comandos a experimentar:

```
2+2
sin(pi/2)
mode
solve equation(x^2-5*x+6)
solver(x+2)
create matrix(foo\2\2\3)
see variables
exit
```

Regras básicas de leitura:

- _ e usado pelo ATC como marcador de negativo em várias formas documentadas

e outputs; por exemplo _5 significa -5.

- #0, #1 e indices seguintes referem resultados anteriores da sessao atual.

- Uma expressão direta e avaliada de imediato, por exemplo 2+2.

- Um comando inicia uma acao ATC, por exemplo `mode` ou `solve equation(...)`.

- Um módulo guiado abre um workflow interativo, por exemplo `financial

calculations ou unit conversions`.

Workflow inicial recomendado:

```
mode
2+2
sin(pi/2)
solve equation(x^2-5*x+6)
verbose resolution(1)
sin(pi/6)
verbose resolution(0)
```

Quando um resultado é importante, confirmar primeiro o modo angular e o modo de precisão.

4. Bases de input

As expressões são escritas diretamente no prompt.

```
2+3*4
sqrt(9)
sin(pi/2)
log(100)
```

Outputs estáveis:

```
#0=14
#1=3
#2=1
#3=2
```

O ATC usa _ como marcador negativo em muitos inputs e outputs documentados:

```
_6+2
```

Resultado esperado:

```
#0=-4
```

Notacao cientifica usa E maiusculo:

```
1E3
1E-3
```

5. Prompt interativo

O ATC 2.1.9 inclui editor de linha:

- Tab completa comandos, funções matemáticas, aliases e funções do

utilizador;

- `Tab` repetido alterna entre completamentos possíveis;

- `Up` e `Down` navegam no histórico;

- `Left`, `Right`, `Home`, `End`, `Delete` e `Backspace` editam a linha atual.

6. Modo angular

Use:

```
mode
```

Opcoes:

```
radian -> 1
degree -> 2
gradian -> 3
```

Exemplos:

```
sin(pi/2)
sin(30)
```

Use `sin(pi/2)` em radianos e `sin(30)` em graus.

7. Precisao

O ATC 2.1.9 suporta:

- `double`

- Boost `mp_float`

Alternancia persistente:

```
higherprecision(1)
higherprecision(0)
```

A definição e aplicada após reiniciar o ATC.

Comandos uteis:

```
dp15dppi
dp50dppi
dp50dpe
dp50dpmaxprecpi
```

Exemplos validados:

```
dp50dppi
3.14159265358979323846264338327950288419716939937511

dp50dpe
2.71828182845904523536028747135266249775724709369996
```

`maxprec` permite usar mais precisão numa expressão sem alterar o modo persistente.

8. Constantes e respostas anteriores

Constantes comuns:

```
pi
e
i
INF
_INF
true
false
```

Resultados anteriores:

```
#0
#1
```

9. Aritmética

Exemplos:

```
2+3*4
(2+3)*4
8/4
2^10
5!
abs(_7)
100rest(3)
100quotient(3)
rtD4D(16)
```

Resultados esperados:

```
2+3*4      -> 14
(2+3)*4    -> 20
2^10       -> 1024
5!         -> 120
abs(_7)    -> 7
rtD4D(16)  -> 2
```

10. Trigonometria

```
sin(pi/2)
cos(0)
tan(pi/4)
sec(0)
cosec(pi/2)
cotan(pi/4)
asin(1)
acos(1)
atan(1)
acosec(1)
asec(1)
acotan(1)
```

Em modo grau:

```
mode
sin(30)
```

Resultado esperado:

```
#n=0.5
```

11. Funcoes hiperbolicas

```
sinh(0)
cosh(0)
tanh(0)
sech(0)
cosech(1)
cotanh(1)
asinh(0)
acosh(1)
atanh(0)
asech(1)
acosech(1)
acotanh(2)
```

12. Logaritmos

```
log(100)
ln(e)
logb2b(8)
```

Resultados esperados:

```
log(100)  -> 2
ln(e)     -> 1
logb2b(8) -> 3
```

13. Numeros complexos

```
2i
```

```
3(2i)
sin(30+30i)
asin(sin(30+30i))
```

O modo angular influencia funções trigonométricas com argumentos complexos.

14. Multiplicacao automática

```
2pi
pi2
2e
2(3+4)
(1+1)(2+3)
2sin(pi/2)
sin(pi/2)2
2i
3(2i)
```

Com variáveis:

```
x=4
2x
x(4)
(4)x
xpi
pix
xe
xsin(pi/2)
```

Quando existe uma variavel com nome exato, o ATC deve preferir essa variavel em vez de dividir o nome em produtos implicitos.

15. Variáveis

```
data=5
data+1
energy=10
energy+1
```

Comandos uteis:

```
see variables
renamed variables
eliminate variables
see results
eliminate results
```

16. Matrizes

Criar matriz:

```
create matrix(foo\2\2\3)
```

Helpers:

```
min(foo)
max(foo)
avg(foo)
linsnum(foo)
colsnum(foo)
getlins(foo)
getcols(foo)
```

Operacoes:

```
foo+foo
foo-foo
foo*foo
foo^2
foo^T
foo^R
foo/2
2*foo
```

17. Estatística e probabilidade

```
min(3\_1\2)
max(3\_1\2)
avg(2\4\6)
```

Resultados:

```
min(3\_1\2) -> -1
max(3\_1\2) -> 3
avg(2\4\6)  -> 4
```

Helpers gaussianos/probabilidade:

```
gerror(0)
gerrorc(0)
gerrorinv(0)
gerrorcinv(1)
qfunc(0)
qfuncinv(0.5)
```

Módulo guiado:

```
statistics calculations
```

18. DSP

```
sinc(0)
sinc(1)
fft(1\0\0\0)
fft(1\2\3)
ifft(1\0\0\0)
ifft(1\1\1\1)
```

19. Polinomios

Simplificacao:

```
simplify polynomial(x)
simplify polynomial(x+1)
simplify polynomial(x+x+1)
simplify polynomial(x^2-5*x+6)
simplify polynomial((x-2)*(x-3))
simplify polynomial((x+1)*(x+1))
simplify polynomial((x-1)^2)
simplify polynomial((x+2)*(x-2))
simplify polynomial((x+2i)*(x-2i))
simplify polynomial(((x-5)(x+2))/(x-5))
```

Raizes para polinómio:

```
roots to polynomial(2\3)
roots to polynomial(2\7\12)
roots to polynomial(6.5i\_6.5i)
roots to polynomial(1+2i\1-2i)
roots to polynomial(0\1\2)
```

Exemplo:

```
roots to polynomial(2\3)
(1+0i)x^2+(_5+0i)x^1+(6+0i)
```

20. Resolucao de equacoes

Quadraticas:

```
solve quadratic equation(1\_5\6)
solve quadratic equation(1\_2\1)
solve quadratic equation(1\2\5)
```

Sistemas:

```
solve equations system(1\1\5;1\_1\1)
```

Equacoes polinomiais:

```
solve equation(x+2)
solve equation(x^2-5*x+6)
solve equation(x^7-12)
solve equation(1\2\3\4\5)
```

```
solve equation((x-1)(x-2))
solve equation(((x-5)(x+2))/(x-5))
solve equation(x^25-100)
```

Exemplos:

```
solve equation(x+2)
x1=-2

solve equation(x^2-5*x+6)
x1=3
x2=2
```

21. Solver

O comando `solver(...)` suporta caminhos de resolução numérica e normalizações rápidas selecionadas. O ATC 2.1.9 valida cada candidato na expressão original antes de o devolver. Uma solução pode ser real ou complexa. Se o caminho anterior do solver não produzir uma raiz válida, é tentado um fallback numérico geral.

```
solver(sin(x)-0.5)
solver(0.5-sin(x))
solver(cos(x)-cos(60))
solver(tan(x)-tan(45))
solver(asin(sin(30))-x)
solver(qfunc(x)-qfunc(0.34233))
solver(x^2-12x-9)
solver(x+2)
solver(((x-5)(x+2))/(x-5))
solver((x-e+pii)(x-e-pii))
solver(x^2-2)
solver(x^2+1)
solver(x^2-2i)
solver(x^2+22)
```

Os últimos quatro exemplos cobrem raízes polinomiais reais e complexas validadas. Quando nenhum candidato satisfaz a expressão original, o ATC apresenta:

```
ATC was unable to find a valid solution.
```

O modo angular e importante nos exemplos trigonometricos.

22. Estudo de funções

```
function study(x+1)
function study(x^2)
function study(x^2-4)
function study(_x^2)
function study(1/(x-1))
function study((x^2-1)/(x-1))
function study((1-x^2)/(x^2-4))
function study(x/(x^2-9))
function study((x+1)/(x^2+1))
```

O estudo pode apresentar dominio, zeros, assintotas, simetrias, sinal, monotonia, concavidade e contradominio quando suportado.

23. Grafico

```
graph settings
graph(x)
graph(x;_2\2\1)
```

O grafico e renderizado na consola. A validação visual completa continua a ser mais adequada manualmente.

24. Modulos guiados

```
triangles rectangles solver
arithmetic matrix solver
financial calculations
geometry calculations
statistics calculations
physics calculations
unit conversions
```

```
microeconomics calculations
```

Estes módulos pedem valores e opções ao utilizador.

25. Receitas por area

Finanças:

```
financial calculations
100*15/100
```

Geometria:

```
geometry calculations
```

Física:

```
physics calculations
```

Conversoes:

```
unit conversions
```

Microeconomia:

```
microeconomics calculations
```

26. Tempo

```
time
calendar(2026)
day of week(y2026m6d20)
day of week(d20m6y2026)
time difference calculations
actual time response
```

Ferramentas como stopwatch, timer, big timer, clock e big clock devem ser usadas manualmente quando envolvem tempo real ou janelas abertas.

27. Ordenacao

```
ascending order(3\1\2)
descending order(3\1\2)
ascending order(_3\1\_2\0)
descending order(_3\1\_2\0)
ascii order
inverse ascii order
```

28. TXT processing e command bridge

```
predefine txt
solve txt
solve txt(name)
open txt("C:\Temp\file.txt")
to solve
enable txt detector
eliminate strings
atc from cmd
atc over cmd
auto solve txt
```

Fluxo tipico:

1. Criar um TXT com um comando ATC por linha. 2. Executar `predefine txt` e indicar o caminho. 3. Executar `solve txt`. 4. O ATC cria um ficheiro `_answers.txt`.

Exemplo de TXT:

```
2+2
sqrt(9)
solver(x-2)
solve equation((x-1)(x-2))
simplify polynomial((x+1)(x-1))
2++
2+3
```

O ATC deve continuar após uma linha invalida e reportar o erro dessa linha.

29. Scripting

O guia online documenta helpers como:

```
print(...)
get(...)
sprint(...)
while
for
if
else
switch
case
cls()
break
return
replace
replace by index
count occurrences
delete x occurrences
is contained
calc
is equal
get positive value
is to write
is variable
is contained variable
is contained by index
strlen
```

Use scripting para fluxos repetiveis e teste scripts pequenos antes de os usar em cálculos maiores.

30. Funcoes do utilizador

As funções do utilizador vivem na pasta `User functions`. O autocomplete pode sugerir funções como `atc_<nome>(`.

31. Settings e ambiente

```
current settings
verbose resolution
verbose resolution(1)
verbose resolution(0)
numerical systems
si prefixes
actual time response
dimensions
window
about
auto adjust window
enable atc intro
disable atc intro
clean history
clean
exit
```

32. Ficheiros e pastas

O ATC gere pastas como:

- pasta de dados ATC;
- Scripts examples;
- Source code;
- Strings;
- To solve;
- User functions.

Alguns comandos abrem Explorer, Notepad ou visualizadores externos.

33. Exportacao

Alguns comandos perguntam:

```
Export result? (Yes -> 1 \ No -> 0)
```

Use `1` para exportar ou `0` para ignorar.

34. Verbose resolution

```
verbose resolution(1)
verbose resolution(0)
```

O modo verbose ajuda a aprender e depurar, mas o uso normal e mais limpo com verbose desligado.

35. Cookbook / Receitas

Documentação online do projeto:

- Cookbook do ATC

Inclui workflows praticos para cálculo cientifico, trigonometria, polinómios, matrizes, estatística, DSP, TXT processing, verbose resolution e módulos guiados.

36. Perfis de utilização

Engenharia:

```
solve equation(x^2-5*x+6)
create matrix(foo\2\2\3)
foo^2
unit conversions
physics calculations
```

Educação:

```
2+3*4
sin(pi/2)
mode
geometry calculations
function study(x^2)
```

DSP:

```
sinc(0)
fft(1\0\0\0)
ifft(1\1\1\1)
sin(pi/2)+cos(0)
```

Estatística:

```
avg(2\4\6)
min(3\_1\2)
max(3\_1\2)
statistics calculations
qfunc(0)
```

Finanças:

```
financial calculations
100*15/100
```

37. Boas práticas

Documentação online do projeto:

- Boas práticas do ATC

Usar para workflows fiáveis: começar por expressões pequenas, confirmar modo angular, validar parenteses, escolher o solver certo, usar TXT para lotes reprodutíveis e evitar comportamento não documentado.

38. Resolucao de problemas

Se uma expressão falhar:

- verificar parenteses;
- verificar modo angular;
- confirmar se deve usar _ para negativo;
- confirmar se o comando e interativo;
- testar partes menores da expressão;
- usar `verbose resolution(1)`;
- confirmar se a funcionalidade esta documentada e coberta por testes.

39. Documentação e testes

Documentação online do projeto:

- Testes

Resultado histórico anterior ao candidato final 2.1.9:

Estes resultados não aprovam o candidato atual. A validação final deve identificar os fontes e os hashes dos executáveis.

```
All: 376 passed, 0 failed
SolverComplex: 14 passed, 0 failed
Txt: 15 passed, 0 failed
Settings: 7 passed, 0 failed
```

Este guia deve evoluir com os testes e com o comportamento documentado do ATC 2.1.9.